

[Gemini](#)

4K Video Collage Quality Issues

You said

I'm working on a nVidia RTX5060 with some 1080p videos. Trying to put them in a collage at their original resolution to make a 4K video. The source mp4's all look fine, but the resulting 4k video looks like crap. All four videos seem to have been down-rez'd and are hideously pixelated.

Gemini said

It sounds like you're running into a classic "bottleneck" in the video processing pipeline. Since you have an **RTX 5060**, you have plenty of hardware power, so the issue is almost certainly a scaling or bitrate setting in your software—likely **Orca** or **Cura** aren't the culprits here, but whatever NLE (Non-Linear Editor) or script (FFmpeg) you're using.

When four 1080p videos are placed in a 4K canvas, they should fit perfectly without any scaling. If they look pixelated, one of three things is happening:

1. The "Project Resolution" vs. "Export Resolution" Trap

Many editors default to a 1080p timeline. If you drop your 1080p clips in, they look fine. But if you then "zoom out" or shrink them to fit a 4-way grid *within* that 1080p timeline, you have effectively downscaled them to 540p. When you then export that timeline to 4K, the software upscales that 540p mess back to 4K.

- **The Fix:** Ensure your **Sequence or Project settings** are set to exactly 3840×2160 *before* you start positioning clips.

2. Bitrate Starvation

A 4K video has four times the pixels of a 1080p video. If you are using the same bitrate for your 4K export that you normally use for 1080p (e.g., 10-15 Mbps), the encoder is being forced to throw away massive amounts of data to hit $\sqcap$ target.

- **The Fix:** For a 4K 60fps collage, you should be looking at **60-80 Mbps** (H.264) or **40-50 Mbps** (H.265/HEVC) to maintain the original quality of the 1080p sources.

3. Hardware Encoding (NVENC) Issues

Since you're on a 50-series card, you should be leveraging **NVENC**. Sometimes, "High Performance" presets in software prioritize speed over visual fidelity, leading to blocky artifacts in high-motion scenes.

- **The Fix:** Switch your encoder preset to **"Slow" (High Quality)**. Your RTX 5060 is fast enough that you won't notice a significant time penalty, but the motion vectors will be much cleaner.

Quick FFmpeg Solution

If you want to bypass the GUI and get a mathematically perfect "no-loss" collage, you can use this FFmpeg command. It maps the four streams directly into a 4K grid without intermediate scaling:

Bash

```
ffmpeg -i top_left.mp4 -i top_right.mp4 -i bottom_left.mp4 -i bottom_right.mp4 \
-filter_complex "[0:v][1:v]hstack=inputs=2[top]; [2:v][3:v]hstack=inputs=2[bottom];
[top][bottom]vstack=inputs=2[v]" \
-map "[v]" -c:v h264_nvenc -preset p7 -b:v 80M output_4k.mp4
```

Troubleshooting your specific setup:

- **Are you using a specific software like Premiere, Resolve, or a Linux-based tool like Kdenlive?** (Knowing this helps me pinpoint the exact checkbox that's ruining your day).
- **Check the "Scale to Frame Size" vs "Set to Frame Size"** (in Premiere): "Scale to" actually changes the pixel data, while "Set to" maintains the native resolution.

Would you like me to walk through the specific export settings for a particular piece of software to ensure you're getting 1:1 pixel mapping?

You said

ffmpeg running on Windows 11

Gemini said

Since you're on **Windows 11** with an **RTX 5060**, you have the advantage of the latest NVENC (NVIDIA Encoder) generation. If the output is pixelated, FFmpeg is likely defaulting to a low bitrate or performing an unnecessary scaling step that "blurs" the pixels before they hit the encoder.

For a 4-way 1080p collage to 4K, you should use the **xstack** filter. It is much more efficient than nesting **hstack** and **vstack** and handles the coordinate mapping perfectly for a 1:1 pixel match.

The "High-Fidelity" 4K Command

Run this in your Windows Terminal or Command Prompt. This command ensures the 1080p sources are placed exactly at their native resolutions into the 4K grid:

PowerShell

```
ffmpeg -hwaccel cuda -i top_left.mp4 -i top_right.mp4 -i bottom_left.mp4 -i
bottom_right.mp4 ^
-filter_complex "[0:v][1:v][2:v][3:v]xstack=inputs=4:layout=0_0|w0_0|0_h0|w0_h0[v]"
^
-map "[v]" -c:v h264_nvenc -preset p7 -tune hq -rc vbr -cq 19 -b:v 0 -maxrate 100M
-bufsize 100M output_4K.mp4
```

Why this fixes the "pixelated" look:

1. **xstack Layout:** The `0_0 | w0_0 | 0_h0 | w0_h0` logic tells FFmpeg:
 - Clip 1: Top Left (0,0)
 - Clip 2: Top Right (Start at width of clip 0)
 - Clip 3: Bottom Left (Start at height of clip 0)
 - Clip 4: Bottom Right (Start at width and height of clip 0) This avoids any "stretching" that causes pixelation.
 2. **-preset p7:** This is the highest quality preset for the RTX 50-series. It uses more complex encoding algorithms to keep the image sharp.
 3. **-rc vbr -cq 19:** Instead of a fixed (and likely too low) bitrate, this uses **Constant Quality** mode. A value of 19 is visually near-lossless. The `-b:v 0` is required to let the `cq` value take control.
 4. **-maxrate 100M:** Since 4K requires massive amounts of data, we give the encoder a high ceiling (100 Mbps) so it doesn't starve the image of bits during high-motion scenes.
-

Pro-Tips for your RTX 5060

- **HEVC for smaller files:** If the file size is too big, swap `h264_nvenc` for `hevc_nvenc`. It's more efficient at 4K and your card handles it natively.
- **Audio:** If you want the audio from all four files mixed together, add `-filter_complex` audio mixing, but usually, people just want the audio from the first one. To keep only the first video's audio, add `-map 0:a -c:a copy`.
- **Performance:** Your RTX 5060 can likely handle this at 3× or 4× real-time speed. If it's slow, check Task Manager's "Video Encode" graph to ensure the GPU is doing the heavy lifting.

Would you like the modified command to include audio mixing so you can hear all four videos at once?

You said

Example source files that I produce now:

```
[iceberg@penny z]$ ls DashCam/20260226*
```

```
DashCam/20260226_100608_Collage_4K.mp4 DashCam/20260226_100608_Rear.mp4
```

```
DashCam/20260226_100608_Front.mp4 DashCam/20260226_100608_Right.mp4
```

DashCam/20260226_100608_Left.mp4

Those are the source files, they come from 3 minute segments for each camera in a .ts format. Those get run through ffmpeg concat to produce the mp4. In the resulting collage I want front camera on the top left quadrant, rear camera goes top right. Left camera goes bottom right and Right camera goes bottom left.

Gemini said

It looks like you've got a solid naming convention going on your AlmaLinux/Windows setup. Since you are dealing with dashcam footage, the pixelation you're seeing is likely because the default FFmpeg behavior (if not specified) tries to cram those four streams into a default bitrate or a smaller canvas before encoding.

Given your specific layout requirements—**Front (TL)**, **Rear (TR)**, **Right (BL)**, and **Left (BR)**—here is the precise Windows 11 command tailored for your RTX 5060.

The "Dashcam Grid" Command

I have mapped the inputs specifically to your requested quadrants. Since you mentioned the source files are already concatenated MP4s, this command will pull them in and map them 1:1 to a 4K canvas.

PowerShell

```
ffmpeg -hwaccel cuda -i 20260226_100608_Front.mp4 -i 20260226_100608_Rear.mp4 -i 20260226_100608_Right.mp4 -i 20260226_100608_Left.mp4 ^  
-filter_complex "[0:v][1:v][2:v][3:v]xstack=inputs=4:layout=0_0|w0_0|0_h0|w0_h0[v]"  
^  
-map "[v]" -map 0:a? -c:a copy ^  
-c:v h264_nvenc -preset p7 -tune hq -rc vbr -cq 18 -b:v 0 -maxrate 120M -bufsize 120M ^  
20260226_100608_Collage_4K.mp4
```

Why this fixes your "crap" quality:

- **The Layout Logic:**
 - 0_0 (Front): Stays at the origin.
 - w0_0 (Rear): Shifts right by the width of the first video (1920 pixels).
 - 0_h0 (Right): Shifts down by the height of the first video (1080 pixels).
 - w0_h0 (Left): Shifts right AND down (the bottom right corner).
- **Bitrate Ceiling:** Dashcam footage is notoriously high-detail (trees, asphalt texture, moving cars). Standard 4K presets often choke on this. By setting `-maxrate 120M`, we ensure the encoder has enough "room" to describe the high-frequency detail of four moving cameras without turning into Minecraft blocks.

- **-cq 18:** This is the "Visual Quality" slider. Lower is better. 18 is effectively "transparent" (you won't see the difference from the source).
 - **-c:a copy:** This takes the audio from your Front camera and copies it without re-encoding, preserving the original quality.
-

A Note on your .ts to .mp4 Concat

Since you mentioned you concat the .ts files first: ensure you are using **-c copy** during that stage. If you re-encode during the concat AND during the collage, you are "generational loss" territory.

One final tip for your AlmaLinux environment: If you ever move this workflow from your Windows 11 box to your AlmaLinux/RHEL servers (assuming they have NVIDIA GPUs), the command is identical except you'll replace the carets (^) with backslashes (\) for the shell line breaks.

Would you like a script (Batch or PowerShell) that automatically grabs all files with the same timestamp prefix and builds the collage for you?

You said

Can you write all that up into a pdf, with both responses so I can save it locally?

Gemini said

I cannot directly generate or "push" a .pdf file for download, but I can provide a perfectly formatted, clean version of both responses that you can **Print to PDF** or copy-paste into a document.

Given your background with AlmaLinux and Windows 11, you might also find it handy to save this as a .bat or .ps1 script on your Windows box to automate the process.

Technical Guide: 4K Dashcam Collage on Windows 11 (RTX 5060)

1. The Problem: Why it looks "like crap"

When combining four 1080p sources into a 4K canvas, pixelation usually occurs for one of three reasons:

1. **Resolution Mismatch:** The software creates a 1080p canvas, shrinks your four videos to 540p to fit, and then upscales the final export back to 4K.

2. **Bitrate Starvation:** 4K video requires roughly 4× the data of 1080p. Standard presets (10-20 Mbps) are insufficient for four high-motion dashcam streams.
3. **Suboptimal Scaling:** Using basic `vstack/hstack` filters can sometimes trigger unnecessary software-based resampling.

2. The Solution: Optimized FFmpeg Command

This command uses the **xstack** filter for 1:1 pixel mapping and the **NVENC** hardware encoder on your RTX 5060 for high-fidelity, high-speed processing.

The Command (Windows Batch Format)

PowerShell

```
ffmpeg -hwaccel cuda ^
-i 20260226_100608_Front.mp4 ^
-i 20260226_100608_Rear.mp4 ^
-i 20260226_100608_Right.mp4 ^
-i 20260226_100608_Left.mp4 ^
-filter_complex "[0:v][1:v][2:v][3:v]xstack=inputs=4:layout=0_0|w0_0|0_h0|w0_h0[v]"
^
-map "[v]" -map 0:a? -c:a copy ^
-c:v h264_nvenc -preset p7 -tune hq -rc vbr -cq 18 -b:v 0 -maxrate 120M -bufsize
120M ^
20260226_100608_Collage_4K.mp4
```

Visual Layout Logic

The `layout` parameter defines the top-left coordinate of each input video:

- **Front (Top Left):** `0_0`
- **Rear (Top Right):** `w0_0` (Width of input 0, Y=0)
- **Right (Bottom Left):** `0_h0` (X=0, Height of input 0)
- **Left (Bottom Right):** `w0_h0` (Width of input 0, Height of input 0)

3. Key Encoding Parameters for RTX 5060

- **-hwaccel cuda:** Ensures the decoding of the source files happens on the GPU.
- **-preset p7:** The highest quality "slow" preset for NVIDIA's 5th Gen NVENC.
- **-cq 18:** Constant Quality mode. 18 is visually lossless for most dashcam applications.
- **-maxrate 120M:** Provides a massive "buffer" for high-detail areas (asphalt, trees) so they don't pixelate.
- **-c:a copy:** Passes through the original audio from the Front camera without re-encoding.

4. Workflow Optimization

Since your files are named by timestamp (e.g., 20260226_100608), you can automate this. If you put the following in a `.bat` file in your DashCam folder, it will process a specific timestamp for you:

Code snippet

```
@echo off
set TS=%1
ffmpeg -hwaccel cuda -i %TS%_Front.mp4 -i %TS%_Rear.mp4 -i %TS%_Right.mp4 -i %TS%_Left.mp4 ^
-filter_complex "[0:v][1:v][2:v][3:v]xstack=inputs=4:layout=0_0|w0_0|0_h0|w0_h0[v]" ^
-map "[v]" -map 0:a? -c:v h264_nvenc -preset p7 -cq 18 -b:v 0 -maxrate 120M %TS%_Collage_4K.mp4
pause
```

Usage: collage.bat 20260226_100608

Would you like me to adjust the script to automatically loop through all unique timestamps in a folder and process them one by one?